

Compilador para uma linguagem simples

Disciplina: Compiladores — 2026/1

Turma: ESW-NOT-PIT-7S-T2 (53334)

Professora: Sheila Tirony

Aluno: Guilherme Gentili

Repositório: github.com/gentilidev/Compiladores

1. Visão geral

Compilador escrito em Python, sem bibliotecas prontas de compilador (sem PLY, ANTLR ou o módulo `re`). Recebe um programa-fonte, passa pelas cinco fases de tradução e executa o resultado em uma máquina virtual de pilha. As fases formam um pipeline, cada uma consumindo a saída da anterior:

```
código-fonte → tokens → AST → AST validada → código de três endereços →  
bytecode → execução
```

Arquivos do projeto:

Arquivo	Função
scanner.py, tokens_def.py, token_model.py	análise léxica
parser.py, ast_nodes.py	análise sintática (AST)
semantic.py	análise semântica (tabela de símbolos e tipos)
ir.py	código intermediário (TAC)
codegen.py	bytecode e máquina virtual
main.py	driver que liga as fases

2. Como rodar

Requer apenas Python 3. Compilar e executar:

```
python3 main.py exemplos/condicional.txt
```

Ver a saída de cada fase (tokens, AST, tabela de símbolos, TAC e bytecode):

```
python3 main.py exemplos/condicional.txt --debug
```

Programas com `read` leem da entrada padrão:

```
echo 5 | python3 main.py exemplos/soma.txt
```

3. As fases

A. Análise léxica

O que faz: transforma o texto em uma lista de tokens (palavras reservadas, identificadores, números, strings e operadores). Espaços e comentários (`//`) são descartados.

Por que funciona: o scanner lê caractere por caractere como um autômato finito determinístico — olha o caractere atual e decide o estado seguinte. Os laços de leitura implementam as expressões regulares: ler enquanto for dígito equivale a `[0-9]+`; ler enquanto for letra ou dígito equivale a um identificador. Ler o máximo que casa (*maximal munch*) garante que `==` seja um token só, e não dois `=`.

B. Análise sintática

O que faz: verifica se a ordem dos tokens respeita a gramática e monta a árvore de sintaxe abstrata (AST). Ordem inválida (ex.: falta de `;`) gera erro com a linha.

Por que funciona: usa parser descendente recursivo — uma função por regra da gramática. A precedência dos operadores está embutida na ordem das chamadas (`igualdade` → `comparação` → `soma` → `produto` → `unário` → `primário`); como `produto` está mais embaixo, `*` e `/` são agrupados antes de `+` e `-`. Por isso `2 + 3 * 4` resulta em 14.

C. Análise semântica

O que faz: garante que o programa faz sentido. Verifica quatro regras: variável declarada antes do uso; sem redeclaração no mesmo escopo; tipos compatíveis nas operações; condição de `if` e `while` do tipo booleano.

Por que funciona: a tabela de símbolos é uma pilha de escopos, cada um um dicionário `nome` → `tipo`. Entrar em um bloco empilha um escopo; sair desempilha. A busca vai do escopo mais interno ao mais externo. A verificação de tipos percorre cada expressão e devolve o tipo dela (`int`, `bool` ou `string`), comparando com o que cada operação exige.

D. Código intermediário (TAC)

O que faz: traduz a AST para código de três endereços — instruções simples com no máximo três operandos (ex.: `t1 = 3 * 4`), independentes de máquina.

Por que funciona: cada expressão devolve o lugar onde seu resultado ficou (uma constante, uma variável ou um temporário), e a operação de cima usa esse lugar. Os comandos `if` e

`while` são convertidos em desvios condicionais com rótulos, que é a forma como o controle de fluxo existe em baixo nível.

```
if (cond) A else B   →   while (cond) C   →
    ifFalse cond goto L1      L1:
    A                        ifFalse cond goto L2
    goto L2                  C
L1: B                    goto L1
L2:                      L2:
```

E. Código final (bytecode) e máquina virtual

O que faz: converte o TAC em bytecode para uma máquina de pilha e executa esse bytecode na VM, produzindo a saída. O enunciado permite Assembly ou bytecode para uma VM; o bytecode foi escolhido porque torna possível executar o programa e mostrar o resultado.

Por que funciona: em uma máquina de pilha, operações empilham operandos e aplicam o operador. `2 + 3` vira `PUSH 2`, `PUSH 3`, `ADD` (desempilha os dois, empilha 5). A VM tem uma pilha para as contas e um dicionário para as variáveis; antes de rodar, mapeia cada rótulo para o índice da instrução, e então executa em laço com um ponteiro de programa, tratando os desvios.

4. A linguagem

- Tipos: `int` e `bool` (`true` / `false`).
- Controle: `if-else` e `while`.
- Aritmética: `+` `-` `*` `/` (divisão inteira) e menos unário.
- Comparação: `==` `!=` `<` `>`.
- Entrada e saída: `read` e `print`.
- Comentários com `//`; strings dentro de `print`.

```
int n;
read n;
int soma;
soma = 0;
int i;
i = 1;
while (i < n + 1) {
    soma = soma + i;
    i = i + 1;
}
print soma;
```

5. Testes

Programas em `exemplos/` e resultados verificados:

Programa	Exercita	Resultado
condicional.txt	if/else, bool, comparação	imprime 10
soma.txt	while, read, aritmética	$n=5 \rightarrow 15$
fatorial.txt	while, string em print	$n=5 \rightarrow 120$
erros.txt	erros semânticos	lista 4 erros e para

Casos adicionais verificados:

- Precedência: $2 + 3 * 4 \rightarrow 14$.
- Erro léxico: caractere inválido (`@`) é rejeitado com a linha.
- Erro de sintaxe: falta de `;` é rejeitada com a linha.
- Divisão por zero: tratada com mensagem de erro, sem quebrar o processo.
- Código de saída diferente de zero em qualquer erro; zero quando executa com sucesso.

6. Limitações

A análise semântica trata escopo corretamente, mas a VM guarda variáveis por nome em um único dicionário; portanto duas variáveis de mesmo nome em escopos diferentes compartilhariam o mesmo espaço na execução. O caso está fora do escopo do trabalho.

7. Conclusão

As cinco fases obrigatórias estão implementadas e a linguagem contém todos os recursos exigidos. O compilador aceita programas válidos, rejeita os inválidos indicando a linha do erro, e o código final executa na máquina virtual com o resultado esperado.